

Doc Linter

The Doc Linter is a standalone, deterministic documentation linter that checks your open Markdown file (the one with focus) against the [Alexa and Appstore Developer Writing Guide](#). Inspired by Vale, with minimal dependencies. Uses [spaCy](#) for NLP-powered sentence analysis.

Requirements

- **Python 3** (pre-installed on Amazon Cloud Desktops)
- **spaCy** with the `en_core_web_sm` model (for mechanically-applied grammar checks):

```
pip3 install spacy
python3 -m spacy download en_core_web_sm
```

How to run the linter

Option 1: Kiro hook (recommended)

1. Open the Markdown file you want to lint in your IDE.
2. In the **Agent Hooks** panel, press the **play** button next to **Doc Linter**.
3. Kiro will prompt you to approve the Python command — select **approve** to run it.
4. The linter results appear in the chat, showing violations for your open file only.

Option 2: Terminal

In Kiro, you can run these commands for more control. In other IDEs that don't use hooks, run these commands from the terminal.

Run from the repo root:

```
# Lint a single file (picks up defaults from linter.yml)
python3 doc-linter/lint.py --format table path/to/file.md

# Only show errors (hide warnings and suggestions)
python3 doc-linter/lint.py --format table --level error path/to/file.md
```

```
# Output as JSON (for CI pipelines or tooling integrations)
python3 doc-linter/lint.py --format json path/to/file.md
```

Configuration

Team-wide linter defaults are set in `doc-linter/linter.yml`. Edit that file to change what the linter reports for everyone. Each setting is documented with inline comments. CLI flags override these settings for individual runs.

What the linter checks

Category	Examples
Terminology	Latin phrases (e.g., i.e., etc.), "utilize", "dev", "platform", "terminal" vs "command prompt"
Voice and tone	Passive voice, "we" pronoun, banned hype words (revolutionary, game-changing), sentence length, banned idioms (under the hood, out of the box), "in order to", missing contractions (do not → don't), sentences starting with "And"
Punctuation	Semicolons, exclamation marks, ampersands in running text
Formatting	Spell out numbers 0–9 (except before units of measurement), repeated words, double articles, OK/ID capitalization, bare callouts (use Liquid includes instead of "Note:", "Warning:", etc.)
Inclusive language	Banned terms (master/slave, whitelist/blacklist, execute, kill)
Accessibility	"click" vs "choose", heading hierarchy, image alt text, no H1, vague pronouns in headings, positional references (above/below)
Word choice	Ambiguous words (once, while, since), jargon (monetize, go-to-market)
Spelling	Dictionary spell checker with suggestions, plus ~150 commonly misspelled words. Skips bold-formatted words, code, <code></code> path content, and proper nouns.
Acronyms	Flags acronyms (3+ uppercase letters) used without being spelled out at first use. Skips a configurable well-known acronyms list.

Category	Examples
Sentence structure	Missing articles before noun phrases, comma splices, sentence fragments, subject-verb agreement, article-vowel agreement (a/an), double-modal detection (e.g., "is also can be")

Spell checker

The spell checker uses a 77,000-word US English dictionary (from [SCOWL](#), the same source used by Vale and Hunspell) supplemented by a technical terms list and a curated proper nouns list (product names, features, and services). Words inside bold formatting, inline code, and Markdown links are skipped. When a misspelling is detected, the linter suggests corrections using a commonly misspelled words list or [edit-distance matching](#).

HTML table support

The linter extracts and checks prose inside HTML `<td>` and `<th>` cells. Cells that contain type signatures, short values (like "Yes", "N/A"), or code-like content are skipped automatically. This means description columns in HTML tables get the same coverage as regular paragraph text.

Acronym introduction check

The linter flags acronyms (3+ uppercase letters) that are used without being spelled out at first use — for example, "MSE" appearing without a prior "Media Source Extensions (MSE)" introduction. It handles:

- Markdown bold/italic around the full name (e.g., `*Vega Developer Tools* (VDT)`)
- Plural forms (e.g., "APIs" maps back to "API")
- Skipping code blocks, HTML comments, table rows, and front matter

Acronyms that the target audience uses daily are listed in `doc-linter/well-known-acronyms.yml` and won't be flagged. Common English words that appear in all-caps (like MUST, TRUE, FALSE from RFC keywords or enum values) are also excluded.

To scan all docs for acronym candidates:

```
python3 doc-linter/scan-acronyms.py
```

This reports acronyms used in 3+ files that are never introduced anywhere — candidates for the well-known list.

The smaller spaCy library, `en_core_web_sm`

The grammar checks (missing articles, subject-verb agreement, etc.) use spaCy's `en_core_web_sm` model (~12 MB) rather than the larger `en_core_web_lg` (~560 MB). The larger model improves dependency-parse accuracy by a few percentage points, but the tradeoff isn't worth it for a docs linter: longer installs, more disk on Cloud Desktops, and slower first-run loading. The small model's main weakness is fixed phrases — it can't tell that "by default" or "call attention to" don't need an article. We handle those with `doc-linter/article-exceptions.yml`, a simple YAML list of phrases the missing-article rule should skip.

Severity levels

In the CLI, you can flag for a specific severity level, otherwise the default uses `suggestion`, which includes them all. Similarly, `warning` includes both warnings and errors, and `error` only includes errors.

- **error** — Hard rule violation. Must be fixed.
- **warning** — Likely a style issue. Should be fixed.
- **suggestion** — Consider revising for clarity.

Adding or editing rules

Rules can be set in `doc-linter/rules/` as YAML files. The linter picks up all `.yml` files in that directory automatically.

Three rule types are supported:

substitution — Flag a word and suggest a replacement:

```
- id: no-utilize
  type: substitution
  category: terminology
  level: error
  message: "Don't use '%s'. Use '%s' instead."
  swap:
    utilize: use
```

existence — Flag a word or phrase if it appears:

```
- id: no-simply
  type: existence
  category: voice
  level: error
  message: "Don't use '%s'."
  tokens:
    - simply
```

pattern — Flag a regex match:

```
- id: passive-voice
  type: pattern
  category: voice
  level: warning
  message: "Consider active voice. Found: '%s'"
  pattern: '\b(is|are|was|were) (used|created)\b'
```